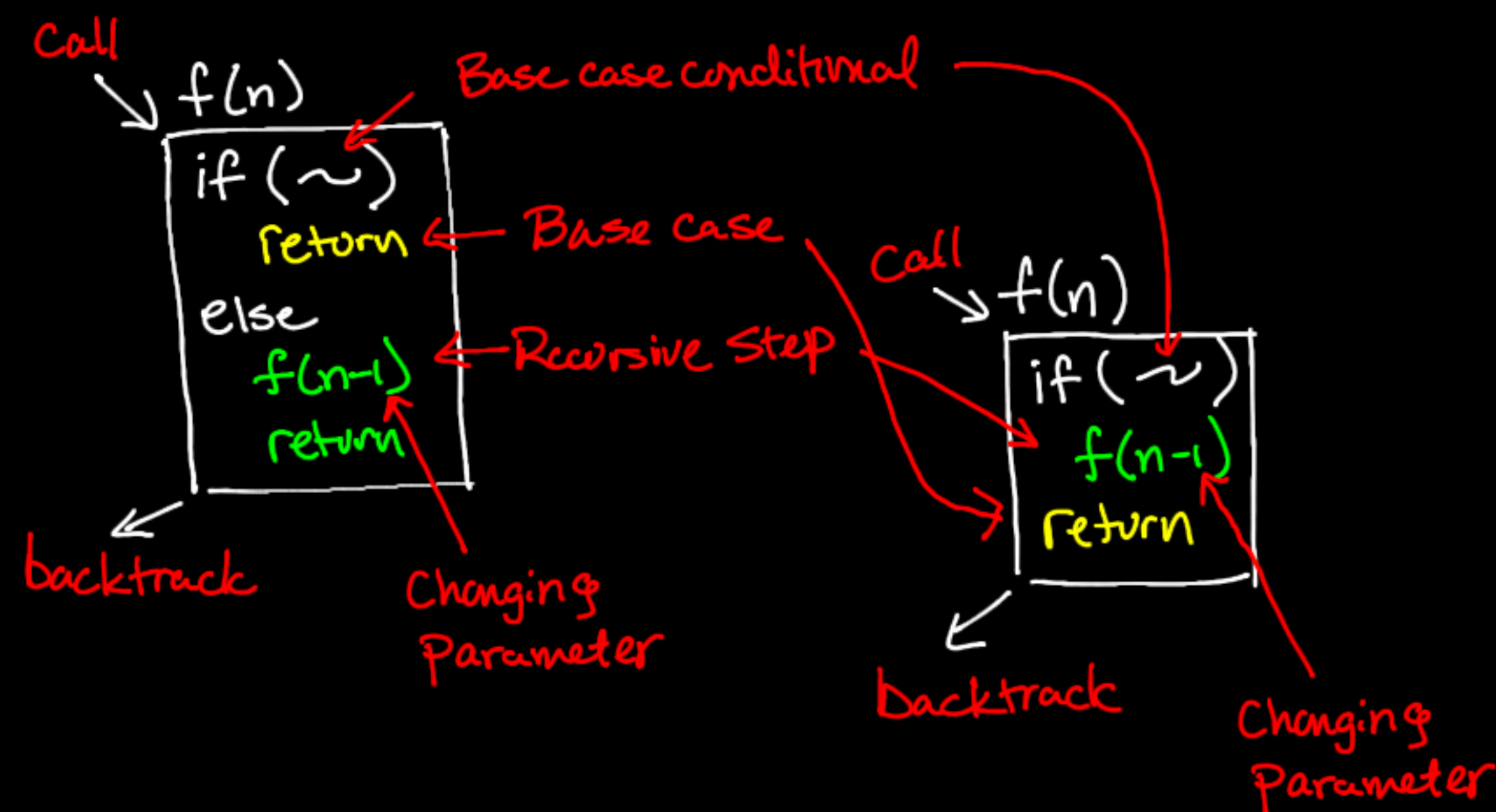
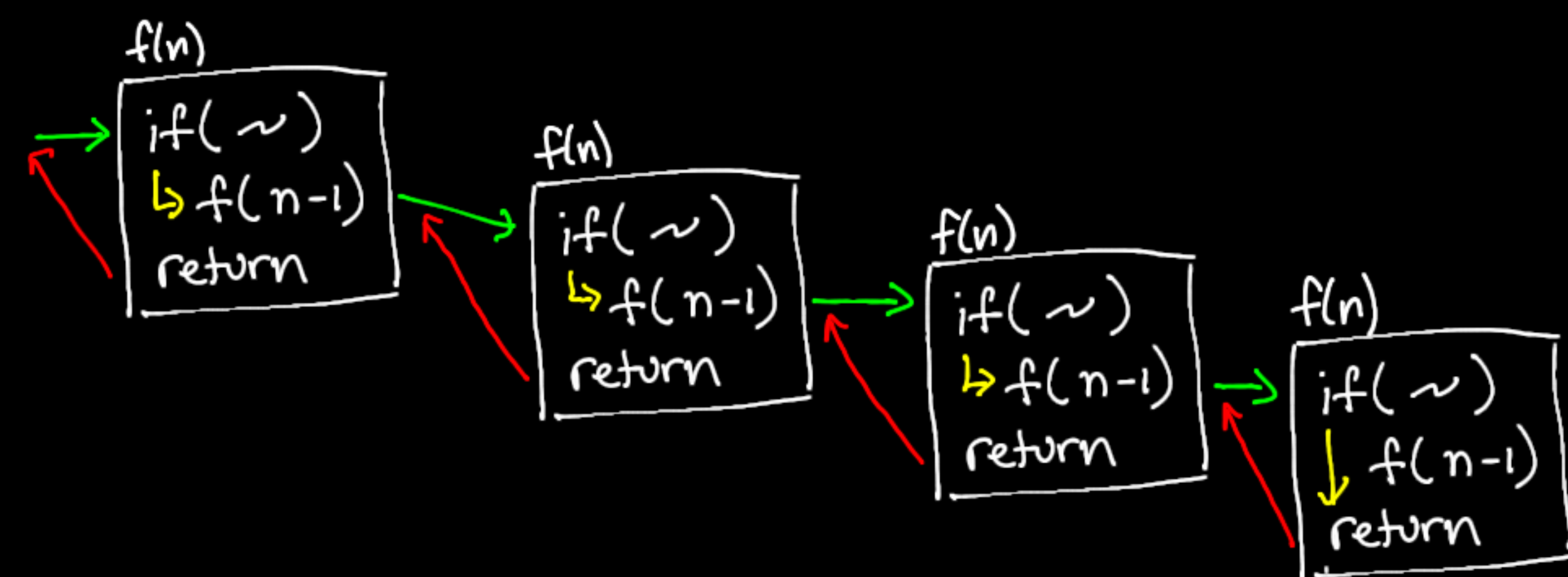


① Box trace



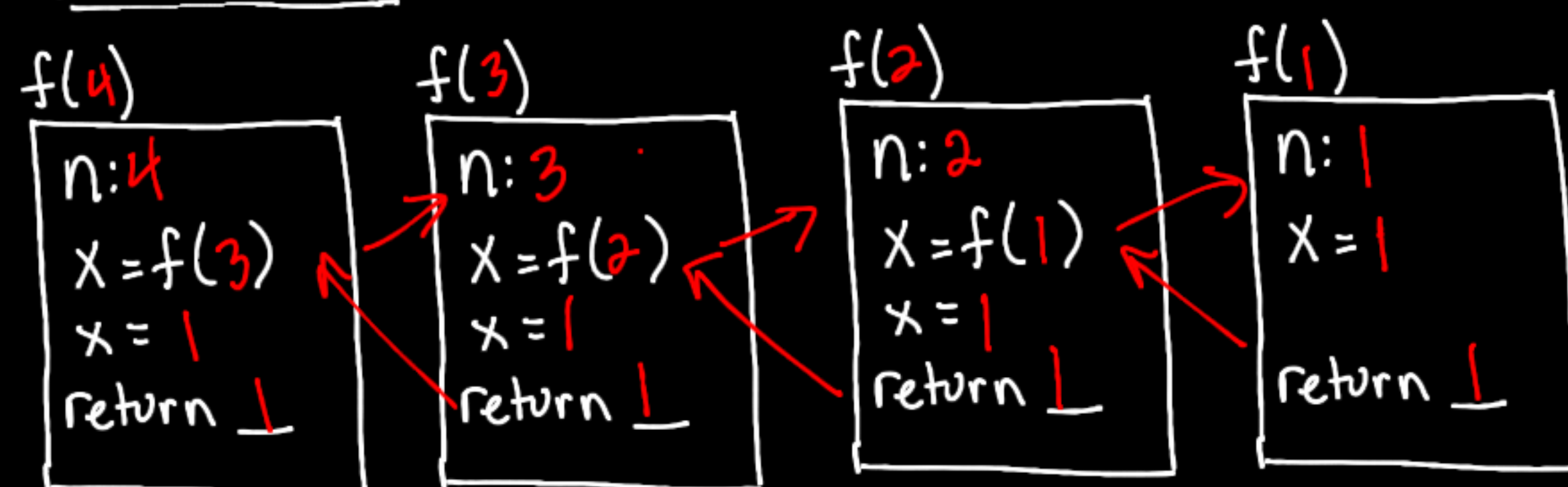
②



③ Box trace w/ local environment

f(n)

```
if (n==1)
  x=n
else
  x=f(n-1)
return x
```



④ Factorials

$$4! = 4 \cdot 3 \cdot 2 \cdot 1$$

Iteration

$$f=1$$

for (n=4, n>0, n--)

$$f = f \cdot n$$

for n>1

$$f(n) = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot (n-(n-1))$$

Other wise

$$f(0)=1 \text{ and } f(1)=1$$

↳ 0!=1 is mathematically define

But also...

$$4! = 4 \cdot \underline{3 \cdot 2 \cdot 1} = 4 \cdot 3!$$

$$3! = 3 \cdot \underline{2 \cdot 1} = 3 \cdot 2!$$

$$2! = 2 \cdot \underline{1} = 2 \cdot 1!$$

$$1! = 1$$

Thus...

$$f(n) = n \cdot [(n-1) \cdot (n-2) \cdot \dots \cdot 1] \\ = n \cdot f(n-1)$$

As a recurrence relation

$$f(n) = \begin{cases} 1 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ n \cdot f(n-1) & \text{if } n>1 \end{cases}$$

⑤ Assuming $n \geq 0$

$$f(n) = \begin{cases} 1, & \text{if } n \leq 1 \\ n \cdot f(n-1), & \text{if } n > 1 \end{cases}$$

$f(4) = 4 \cdot f(3)$ **recurse**
 $f(3) = 3 \cdot f(2)$ **recurse**
 $f(2) = 2 \cdot f(1)$ **recurse**
 $f(1) = 1$ **base case**
 $f(2) = 2 \cdot 1 = 2$ **backtrack**
 $f(3) = 3 \cdot 2 = 6$ **backtrack**
 $f(4) = 4 \cdot 6 = 24$ **backtrack**

- ① f calls itself
- ② parameter n is changing
- ③ f has a base case
- ④ base case is always reached

```

int factorial(int n) {
    if (n <= 1)
        return 1;
    else
        return n * factorial(n-1);
}

```

⑥ Box Trace Club Rules

~~1. You do not talk about Box Trace Club~~

1. Label each recursive call in the code

2. Represent a called method w/ a box containing the local environment

3. Draw a labeled arrow from caller to the called

4. Execute the method w/ its local environment

5. Draw backtrack arrows and fill in the values

~~6. Smile your enemies~~

⑦

